Telegram or Discord bots with a **simple RAG or image-description system.**

🎯 **Goal**

Build a lightweight GenAI bot that can:

- Receive user messages via Telegram (or Discord),
- Perform one of the following:
  - **Option A (RAG):** Retrieve relevant answers from a small knowledge base of documents, or
- Reply back to the user with the generated answer or image caption.

📦 **Assignment Details**

🧩 **1. Bot Interface**

- Choose **Telegram** (using python-telegram-bot) or **Discord** (using discord.py or interactions.py).
- The bot should respond to commands:
  - /ask <query> — for text or RAG queries
  - /image — user uploads an image for description
  - /help — show usage instructions

🧩 **2. Knowledge or Image System**

Pick one of the two tracks:

🔷 **Option A — Mini-RAG**

- Use 3–5 short Markdown or text documents (e.g., company policies, recipes, tech FAQs).
- Build a **retrieval system**:
  - Split into chunks
  - Embed using a local model (e.g., snowflake-arctic-embed, all-MiniLM-L6-v2)
  - Store embeddings in a local SQLite or sqlite-vec DB
  - At query time:
    - Retrieve top-k chunks
    - Build a context + prompt
    - Call a small LLM (Ollama, Hugging Face model, or OpenAI API if preferred)
  - Send back summarized answer to user.

🧩 **3. Optional Enhancements**

(Candidates aiming higher can implement these)

- Message history awareness — maintain last 3 interactions per user.
- Basic caching (don't re-embed already seen queries).
- "Source snippets" — show which doc was used in RAG response.
- /summarize command — summarize last image or chat.

📦 **Deliverables**

- Source code (Python preferred).
- README.md explaining:

    - How to run locally (python app.py or Docker Compose)
    - Which models and APIs are used
    - System design diagram (optional but appreciated)
- Demo screenshots or short GIF of working interaction.

🧪 Evaluation Criteria

Area
What We Look For
Code Quality
Readable, modular, minimal dependencies
System Design
Logical architecture, clear data flow
Model Use
Good reasoning for chosen model (local vs API)
Efficiency
Sensible caching, small model footprint
User Experience
Clear responses, fast turnaround
Innovation
Bonus: multi-modal support or clever prompt design

🚀 Tech Stack Recommendations

- **Bot:** python-telegram-bot or discord.py
- **Embeddings:** sentence-transformers or sqlite-vec